

Memoria Técnica p2:

Evolución a Sistema de Objetos Distribuidos con Java RMI

Juan Palma Prieto – Leonardo Rivas Martín

1. Diseño de la Interfaz Remota y Decisiones Adoptadas

El objetivo principal de esta práctica ha sido evolucionar el sistema cliente-servidor basado en sockets (p1) hacia un modelo de objetos distribuidos utilizando Java RMI. El núcleo de este rediseño es la interfaz ServicioPETD, la cual hereda de java.rmi.Remote y define el contrato de comunicación.

1.1. Nivel de Abstracción

Se ha optado por un nivel de abstracción medio-alto. En lugar de exponer operaciones de bajo nivel, la interfaz refleja a la perfección el caso de uso del sistema de tareas distribuidas (PETD) acordado previamente. Los métodos expuestos son **login**, **pushTarea** y **pollTarea**. Esto garantiza una alta expresividad, permitiendo que el cliente interactúe con el servidor como si invocara métodos de un objeto local.

1.2. Decisiones de Diseño: Estado Compartido y Concurrencia

Siguiendo las indicaciones del anexo de la práctica, se ha implementado el **Patrón Singleton** para el servidor. En el RMI Registry se registra una única instancia de ServidorRMI. Esto implica que todos los clientes acceden concurrentemente al mismo objeto y comparten el estado (en este caso el GestorCola).

Para garantizar la integridad de los datos frente a accesos simultáneos, se ha reutilizado el modelo de dominio original, el cual ya empleaba estructuras seguras para hilos (ConcurrentHashMap y LinkedBlockingQueue).

1.3. Gestión de Sesiones (El problema del estado)

Java RMI es, por naturaleza, una herramienta sin estado. A diferencia de los Sockets, donde un hilo mantenía viva la conexión del cliente, en RMI una invocación a **login** es independiente de un posterior **pushTarea**. Para mantener la semántica de las operaciones y la seguridad, se ha implementado un mecanismo que utiliza RemoteServer.getClientHost() para extraer la IP del cliente remoto. Las IPs autenticadas se almacenan en un HashSet en el servidor, validando dinámicamente si un cliente tiene permisos para realizar operaciones de escritura (push).

2. Reflexión Técnica

El cambio de escenario tecnológico de Sockets TCP a Invocación Remota de Métodos (RMI) ha transformado radicalmente la arquitectura del software, demostrando que los patrones de comunicación trascienden la implementación.

2.1. Transformación del protocolo en métodos

El protocolo original se basaba en el intercambio explícito de mensajes de texto estructurados (ej. "**LOGIN admin 1234**" o "**PUSH FACTORIAL 5**"). El programador debía construir, enviar, recibir y analizar estas cadenas.

En esta evolución, **el protocolo explícito se ha transformado en invocación implícita**. Las reglas del protocolo original son ahora las firmas de los métodos de la interfaz Java. Los parámetros (usuario, contraseña, tipo de tarea) viajan como variables tipadas. Además, las respuestas ya no son texto plano, para la consulta de estado, el método **pollTarea** devuelve el objeto Tarea completo, serializado nativamente por la JVM gracias a la interfaz `java.io.Serializable`.

2.2. Problemas que han desaparecido respecto a Sockets

La abstracción de la comunicación remota ha eliminado gran parte de la complejidad de infraestructura:

- **Gestión manual de hilos:** Ha desaparecido la clase `ManejadorCliente`. Ya no es necesario crear un `Thread` bloqueante por cada cliente conectado; RMI gestiona sus propios hilos internamente.
- **Gestión de Sockets y Flujos:** Se elimina el manejo de `ServerSocket`, `Socket`, y la apertura/cierre de flujos `BufferedReader` y `PrintWriter`.
- **Análisis sintáctico (Parsing):** Desaparecen los errores derivados de formatear mal los *strings* (ej. espacios en blanco adicionales...) y las excepciones tipo `ArrayIndexOutOfBoundsException` al hacer `split()` del texto recibido.

2.3. Nuevos problemas que han aparecido

Aunque RMI oculta la red, esta abstracción trae consigo nuevos problemas:

- **Latencia y Excepciones Remotas:** Una llamada a método en RMI requiere serialización, viaje por red y deserialización. La red puede fallar en cualquier momento, lo que obliga a declarar `throws RemoteException` en toda la interfaz y a implementar bloques `try-catch` en el cliente para evitar roturas.
- **Fallos de Infraestructura (Interfaces de Red):** En entornos con múltiples interfaces, RMI puede negociar direcciones IP incorrectas para los stubs. Esto requirió intervenir la ejecución inyectando la propiedad `java.rmi.server.hostname` para forzar el uso de la IP real de la red local.

- **Excepciones de Lógica Distribuidas:** Se ha implementado un control capturando `IllegalArgumentException` lanzadas por el servidor (ej. "Acceso Denegado" por falta de login) y mostrándolas en el cliente.

3. Instrucciones de Ejecución (Script)

Para demostrar el correcto funcionamiento distribuido entre dos máquinas reales (**Máquina A - Servidor** y **Máquina B - Cliente**), se deben seguir estos pasos:

Requisitos previos:

- Ambas máquinas deben tener Java 17+ instalado y conexión de red local habilitada.
- Clonar el proyecto Maven (PETD_RMI) en ambas máquinas.

Paso 1: Ejecución del Servidor (Máquina A)

1. Identificar la IP local de la máquina A (ej. 192.168.56.103 en mi caso).
2. En la clase `ServidorRMI.java`, verificar que la propiedad coincide:
`System.setProperty("java.rmi.server.hostname", "192.168.56.103");`
3. Compilar y ejecutar la clase `ServidorRMI`. El registro se levantará en el puerto 1099.

Paso 2: Ejecución del Cliente (Máquina B)

1. En la clase `ClienteRMI.java`, configurar la IP del servidor objetivo:
`LocateRegistry.getRegistry("192.168.56.103", 1099);`
2. Ejecutar `ClienteRMI.java`.
3. Interactuar mediante los comandos interactivos, comenzando por: LOGIN admin 1234.

4. Anexos → Demostración de Ejecución Distribuida

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
tcp.stream eq 0						
No.	Time	Source	Destination	Protocol	Length	Info
8	38.194094933	192.168.56.104	192.168.56.103	TCP	74	37666 → 1099 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1...
9	38.194126670	192.168.56.103	192.168.56.104	TCP	74	1099 → 37666 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460...
10	38.194634700	192.168.56.104	192.168.56.103	TCP	66	37666 → 1099 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=16424055...
11	38.196169613	192.168.56.104	192.168.56.103	RMI	73	JRMI, Version: 2, StreamProtocol
12	38.196172581	192.168.56.103	192.168.56.104	TCP	66	1099 → 37666 [ACK] Seq=1 Ack=8 Win=65280 Len=0 TSval=30444593...
13	38.202966326	192.168.56.103	192.168.56.104	RMI	87	JRMI, ProtocolAck
14	38.203556988	192.168.56.104	192.168.56.103	TCP	66	37666 → 1099 [ACK] Seq=8 Ack=22 Win=64256 Len=0 TSval=1642405...
15	38.203825435	192.168.56.104	192.168.56.103	RMI	81	Continuation
16	38.211044662	192.168.56.104	192.168.56.103	RMI	122	JRMI, Call
17	38.211223582	192.168.56.103	192.168.56.104	TCP	66	1099 → 37666 [ACK] Seq=22 Ack=79 Win=65280 Len=0 TSval=304445...
18	38.229084902	192.168.56.103	192.168.56.104	RMI	382	JRMI, ReturnData
19	38.270546830	192.168.56.104	192.168.56.103	TCP	66	37666 → 1099 [ACK] Seq=79 Ack=338 Win=64128 Len=0 TSval=16424...
31	38.287933540	192.168.56.104	192.168.56.103	RMI	67	JRMI, Ping
32	38.288220203	192.168.56.103	192.168.56.104	RMI	67	JRMI, PingAck
33	38.288567773	192.168.56.104	192.168.56.103	TCP	66	37666 → 1099 [ACK] Seq=80 Ack=339 Win=64128 Len=0 TSval=16424...
34	38.288781827	192.168.56.104	192.168.56.103	RMI	81	JRMI, DgcAck
36	38.339296831	192.168.56.103	192.168.56.104	TCP	66	1099 → 37666 [ACK] Seq=339 Ack=95 Win=65280 Len=0 TSval=30444...
51	68.260269034	192.168.56.104	192.168.56.103	TCP	66	37666 → 1099 [FIN, ACK] Seq=95 Ack=339 Win=64128 Len=0 TSval=...
52	68.261368053	192.168.56.103	192.168.56.104	TCP	66	1099 → 37666 [FIN, ACK] Seq=339 Ack=96 Win=65280 Len=0 TSval=...
53	68.262044859	192.168.56.104	192.168.56.103	TCP	66	37666 → 1099 [ACK] Seq=96 Ack=340 Win=64128 Len=0 TSval=16424...
Frame 13: 87 bytes on wire (696 bits), 87 bytes captured (696 bits) on interface enp0s3, id 0						
Ethernet II, Src: PcsCompu_b9:7f:e4 (08:00:27:b9:7f:e4), Dst: PcsCompu_bb:c5:0e (08:00:27:bb:c5:0e)						
Internet Protocol Version 4, Src: 192.168.56.103, Dst: 192.168.56.104						
Transmission Control Protocol, Src Port: 1099, Dst Port: 37666, Seq: 1, Ack: 8, Len: 21						
Java RMI						
0000	08 00 27 bb c5 0e 08 00 27 b9 7f e4 08 00 45 00	E:				
0010	00 49 2d a4 40 00 40 06 1a eb c0 a8 38 67 c0 a8	.I-@.@....8g.				
0020	38 68 04 4b 93 22 ac 51 27 7b 1e d7 3d 7e 80 18	8h.K"Q '[.-s...				
0030	01 fe f2 5b 00 00 01 01 08 0a 12 25 79 f5 61 e5	...[.....%y.a				
0040	1e b0 4e 00 0e 31 39 32 2e 31 36 38 2e 35 36 2e	..N...192.168.56.				
0050	31 30 34 00 00 93 22	104..."				

Fig1: cap wireshark en la máq Servidor

Captura de tráfico con Wireshark demostrando la ejecución distribuida entre el Cliente (192.168.56.104) y el Servidor (192.168.56.103).

Se observa la conexión al puerto 1099, la negociación del protocolo JavaRMI y el intercambio de paquetes Call/ReturnData correspondientes a la invocación remota de los métodos.